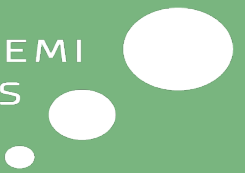
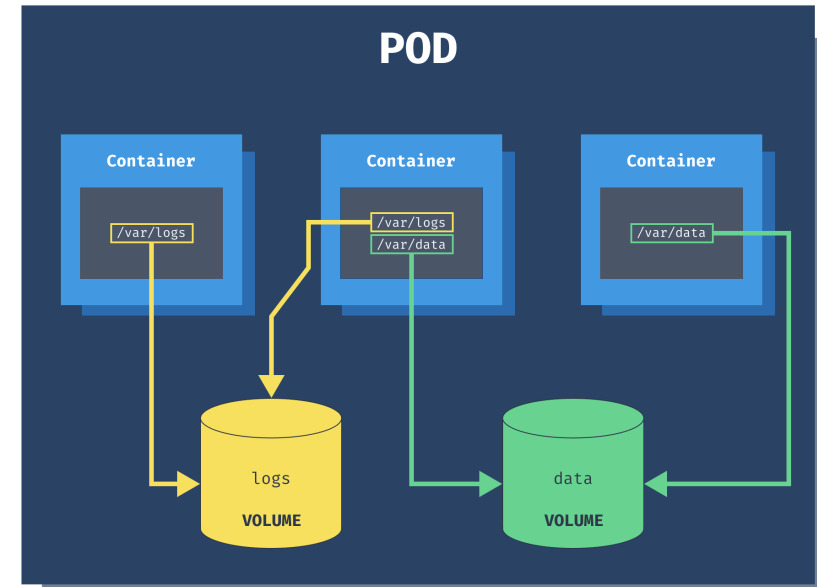


Volumes i K8s



Hvad er en Volume?

- Grundlæggende et filsystem, der er tilgængeligt for en eller flere containere i en Pod.
- Giver mulighed for at dele data mellem containere i samme Pod.
- Abstraherer det underliggende storage-system fra containerne.



Kortvarige (efemere/ephemeral) vs. Persistente

Efemere Volumes: Har samme livscyklus som Pod'en.
Data går tabt når Pod'en slettes.

- er nyttige til midlertidig *storage*, deling af data inden for en Pods levetid, f.eks. *emptyDir*, *configMap* og *secret*.

Persistente Volumes (PV): Designet til at overleve Pod'ens livscyklus.

- Data bevares selvom Pod'en genstartes eller slettes.

(mere på følgende slides)

Vi vil primært fokusere på persistente løsninger her!

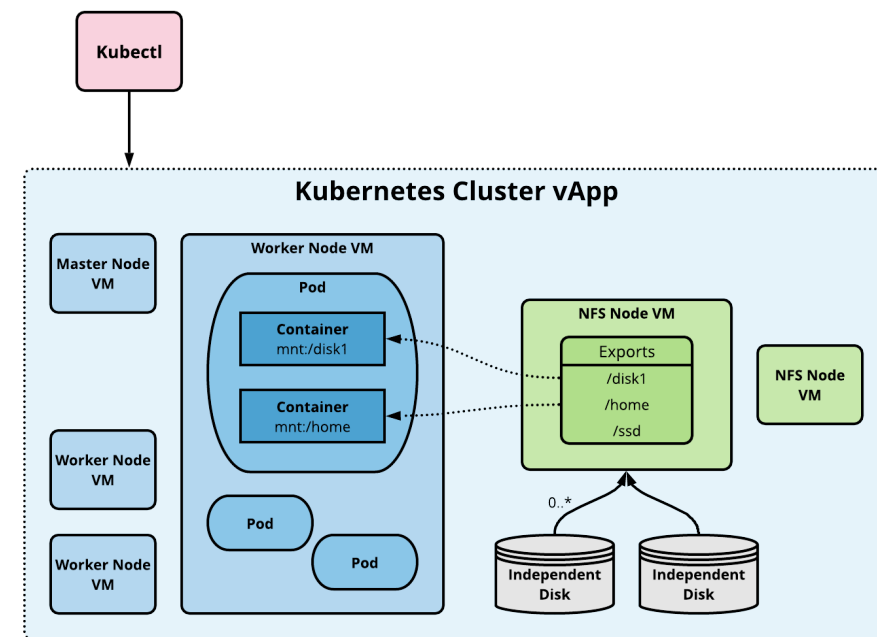
Repræsenterer lokalt tilgængeligt opbevaring på en "worker node" (f.eks. en disk direkte tilsluttet noden).

- **Fordele:**
 - **Høj ydeevne:** Lav latency og høj IOPS, da data er lokalt.
 - **Omkostningseffektivt:** Kan udnytte eksisterende lokal opbevaring.
- **Ulemper:**
 - **Tilgængelighed:** Data er bundet til den specifikke node. Hvis noden fejler, er data utilgængelige.
 - **Skalering:** Svært at skalere på tværs af flere noder.
 - **Administration:** Kan være mere komplekst at administrere.
- **Anvendelse:**
 - Ydeevnekritiske applikationer som ***distribuerede filsystemer*** eller ***NoSQL databaser*** med indbygget replikering.

Network File System (NFS) Volumes

Giver mulighed for at **forbinde** et filsystem fra en ekstern **NFS server** ind i en Pod.

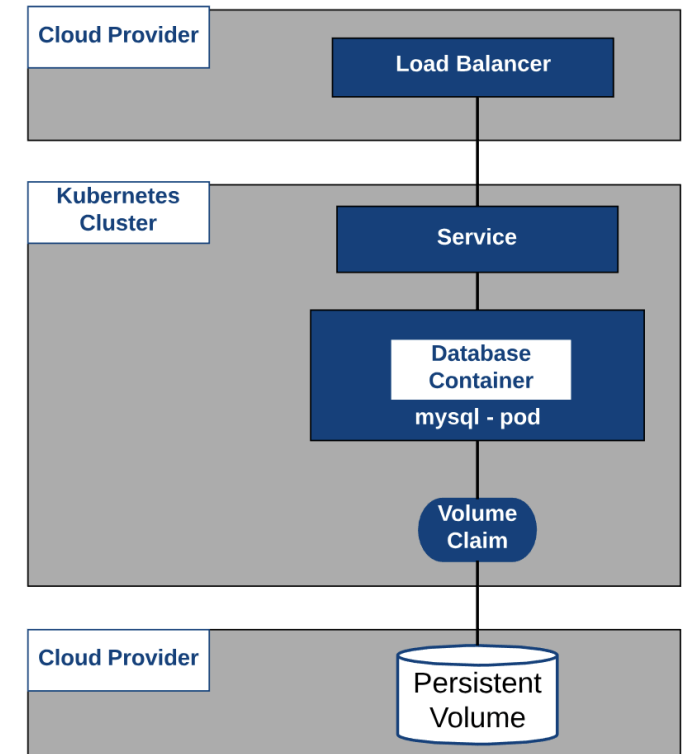
- **Fordele:**
 - **Simpel at sætte op:** Hvis der allerede er en NFS server tilgængelig.
 - **Centraliseret storage:** Data er gemt centralt og kan deles på tværs af flere noder og Pods.
 - **Tilgængelighed:** Afhænger af NFS serverens redundans.
- **Ulemper:**
 - **Ydeevne:** Kan være lavere latency og IOPS sammenlignet med lokal storage, afhængigt af netværksforbindelsen og NFS serverens ydeevne.
 - **Single Point of Failure:** NFS serveren kan være et single point of failure, medmindre den er sat op med høj tilgængelighed.
- **Anvendelse:**
 - Deling af konfigurationsfiler, logfiler, statiske aktiver på tværs af applikationer.
 - Enklere persistens for mindre kritiske applikationer.



Cloud Provider Volumes

Beskrivelse: Integrerer med storage-løsninger fra cloud udbydere (Amazon EBS, Azure Disks, Google Compute Engine Persistent Disks osv.).

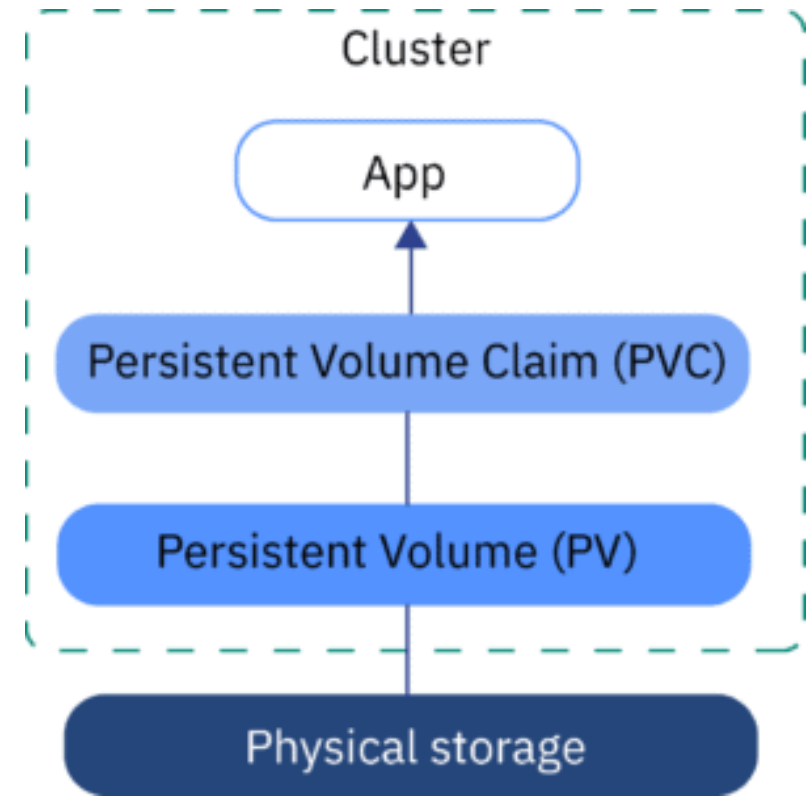
- **Fordele:**
 - **Høj tilgængelighed og redundans:** Typisk replikeret og administreret af en *cloud-provider*.
 - **Nem skalering:** Størrelsen kan ofte justeres dynamisk.
 - **God integration:** Problemfri integration med andre cloud services.
- **Ulemper:**
 - **Vendor Lock-in:** Er bundet til den specifikke cloud provider.
 - **Omkostninger:** Kan være dyrere end lokal storage eller self-hosted NFS.
- **Anvendelse:**
 - Stateful applikationer i cloud miljøer (databaser, message queues osv.).
 - Applikationer der kræver høj tilgængelighed og nem skalering.
 - Dynamisk provisionering via StorageClasses er almindeligt her.



Persistente Volumes: Vedvarende Data i Kubernetes

Udfordringen med standard Volumes: Data i standard volumes forsvinder når Pod'en slettes, hvilket er et problem for stateful applikationer (databaser, osv.).

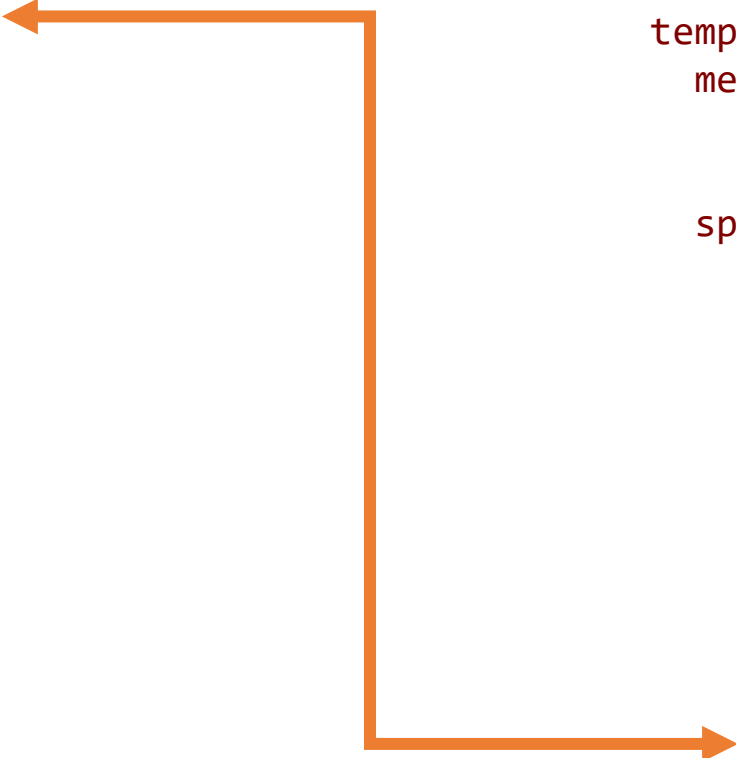
- **PersistentVolume (PV):**
 - En ressource i Kubernetes, der leveres af en administrator eller dynamisk via [StorageClasses](#).
 - Udgør noget storage i clusteret.
 - Har en livscyklus **uafhængig** af individuelle Pods.
 - Kan leveres på forskellige måder (f.eks. lokal storage, netværksstorage som NFS, cloud provider storage).
- **PersistentVolumeClaim (PVC):**
 - En anmodning fra en bruger om persistent storage.
 - En Pod bruger en PVC til at anmode om og få adgang til en PV.
 - Kubernetes finder en passende PV, der matcher PVC'ens krav (størrelse, adgangstilstand).
- **Abstraktion:** PV og PVC adskiller storage-administration fra applikationsudvikling. Udviklere anmoder om storage uden at skulle bekymre sig om den underliggende infrastruktur.



Eksempel: Brug af Persistent Volume med SQLite

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: sqlite-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sqlite-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sqlite-app
  template:
    metadata:
      labels:
        app: sqlite-app
    spec:
      containers:
        - name: sqlite-container
          image: busybox:latest
          command: ["/bin/sh", "-c", "sleep 3600"]
          volumeMounts:
            - name: sqlite-data
              mountPath: /data
      volumes:
        - name: sqlite-data
          persistentVolumeClaim:
            claimName: sqlite-pvc
```



Opgave M11.01

Canvas opgaver

K8s Volumes



Gruppe